Assumptions

Assumptions and design decisions

Assessment requirements implemented

- Account identifiers are unique, generated UUIDs. The client cannot choose an ID, balance, or timestamp.
- New accounts always start with a zero balance.
- Amounts use exact decimal arithmetic and are stored to two decimal places; the interface presents USD. Currency conversion and multi-currency balances are out of scope.
- Transaction IDs and timestamps are generated by the server.
- Credits/debits must be positive; transaction references are required and unique within an account.
- Duplicate transaction requests are idempotent: return the existing transaction and do not post a second balance change.
- Entries are immutable through the API. The database enforces account/reference uniqueness, valid transaction types, positive amounts, and supported account-balance bounds.
- Transaction and balance writes use database transactions; account rows are locked for posting where supported by the configured database. SQLite does not implement row-level SELECT ... FOR UPDATE, so MySQL/InnoDB is required when deployment correctness depends on concurrent row locking.
- History is ordered newest-first and supports type filtering and pagination.
- Money transfers are implemented as two ledger entries inside one atomic operation. Invalid/self transfers are rejected; insufficient funds leave both accounts unchanged.

Additional decisions where the assessment was not specific

- Currency: USD display and a two-decimal precision model are chosen for the UI; the API does not include a currency code.
- Decimals: Maximum transaction amount precision is 14 total digits and 2 fractional digits. Account balances permit 16 total digits and 2 fractional digits.
- Balance ceiling: A credit or transfer that would exceed the maximum representable account balance is rejected with 409 instead of allowing a database precision error.
- Description: Optional, up to 255 characters; empty string when omitted.
- Reference ID: Trimmed, case-sensitive text up to 100 characters. Uniqueness is scoped to an account, not global.
- Duplicate response: 200 OK with duplicate: true; the original record is returned. Conflicting request details do not change the original transaction or balance.
- Transfer duplicate scope: The source account reference determines an idempotent retry. Destination references are also constrained by the account-level unique key, so a pre existing destination reference causes the atomic transfer to fail rather than overwrite existing history.
- Pagination: Default page size 10, maximum 100. Page and limit must be positive integers. Out-of-range pages return 400, rather than silently returning another page.
- Missing accounts: Balance/history paths return 404. Transfer body references to missing accounts return 400.
- Error format: DRF errors are returned in { "error": { "code", "message", "details" } }.
- Timezone: Server timestamps are timezone-aware UTC.

- Database configuration: SQLite is the default for quick local development and automated tests. Relative SQLite DBNAME paths are anchored to the backend directory, and the example points to backend/db.sqlite3. Set DBENGINE=mysql and the DB values to use MySQL; MySQL/InnoDB is required for production row-level locking behavior.
- Authentication/authorization: Authentication was not specified and is not included. Do not expose this assessment app to untrusted users or the public internet without adding authentication, authorization, TLS, and operational safeguards.
- Frontend identity: Workspace administrator labels are generic presentation, not authenticated user data.
- Dashboard rollups: Aggregate totals and chart data are computed server-side across the complete ledger and returned by the dashboard API; no fabricated ledger records are used. • Transaction sorting: The global feed defaults to newest-first and supports oldest-first and amount ascending/descending as explicit sort choices.